Master Upgrade Prompt: Jijiga Express Super-App

You are a senior product architect, UX designer, full-stack engineer, security engineer, and QA lead. Improve the existing **Jijiga Express** project in the attached repository. Do not rebuild the application from scratch and do not replace working features with mockups.

The goal is to make Jijiga Express a polished, production-ready regional super-app that is easier and faster for customers to use than the current experience, while preserving and strengthening the platform's existing logistics, security, dispatch, tracking, inventory, accounting, and specialized-service capabilities.

Use the existing codebase, types, services, Firestore rules, roles, tests, and data models as the starting point. Inspect the project before making changes. Reuse existing components and patterns where appropriate. Make incremental, well-structured changes.

---

1. Product Direction

Jijiga Express must become a trusted regional super-app for Ethiopia and nearby Somaliland markets.

The app should combine:

- Food and restaurant delivery
- Grocery and local-market delivery
- Parcel delivery
- Taxi, Bajaj, and moto booking
- Hotel booking
- EEU prepaid electricity recharge
- Vendor/Mamila marketplace
- Courier dispatch and live tracking
- Customer support and Help Center
- Loyalty points and rewards

The product must have a clear identity of its own. Do not copy BANA or Marhaba branding, logos, text, images, or proprietary design. Use their visible product patterns only as inspiration for better usability.

---

2. Existing Capabilities That Must Be Preserved

Do not remove, weaken, or bypass these existing strengths:

1. Customer, Mamila/Vendor, Runner, Rider, and Admin roles.
2. Firebase Authentication and Firestore data isolation.
3. Firestore security rules and server-side authorization.
4. Order status transition rules.
5. Inventory reservation and stock restoration behavior.
6. Runner verification and Rider dispatch workflow.
7. Admin reassignment of orders.
8. Live route maps, courier coordinates, ETA, and delivery tracking.
9. Delivery-event logging and auditability.
10. Parcel booking and parcel validation.
11. Taxi/Moto ride booking.
12. EEU prepaid-card custody state machine and receipt protection.
13. Dispute, cancellation, refund, and defect-handling flows.
14. Financial accounting and merchant/platform revenue separation.
15. Existing chat, WhatsApp, voice-note, notification, and support capabilities.
16. English, Somali, and Amharic localization foundations.
17. Existing regression tests and security tests.

If a new feature conflicts with an existing security rule or accounting rule, redesign the feature rather than weakening the rule.

---

3. Required Product Experience

3.1 Mobile-first home screen

Redesign the customer home screen to be easy to understand within five seconds.

The home screen must include:

- Current delivery location selector
- Notification button
- Customer name or avatar

- Points balance and tier, once loyalty is enabled
- A compact service grid with recognizable icons
- Promotional banner area
- Popular restaurants and vendors
- Featured products or offers
- Active order or booking card when applicable
- Quick access to Help Center and WhatsApp support

Organize the service grid into understandable groups:

- **Move:** Taxi, Bajaj, Moto, Bus
- **Deliver:** Food, Parcel, Grocery
- **Stay and Services:** Hotel, Business
- **More:** Electricity, Marketplace, Support

Use responsive mobile bottom navigation with:

- Home
- Search or Services
- Orders/History
- Help
- Profile

The desktop layout may use a wider navigation pattern, but the mobile experience must prioritize thumb-friendly bottom navigation.

3.2 Phone-first onboarding

Add a phone-number authentication and onboarding flow suitable for the target market.

Requirements:

- Support Ethiopia `+251` and Somaliland `+252` country codes.
- Validate phone-number format.
- Send a six-digit OTP through the configured provider.
- Display six OTP fields or an equivalent accessible OTP input.
- Include resend countdown.
- Include Change phone number.
- Rate-limit OTP requests and failed attempts.

- Expire OTPs after a configurable period.
- Require Terms and Privacy consent.
- Create only a customer profile during self-registration.
- Never allow customer registration to create admin, vendor, runner, or rider permissions.
- Allow optional email linking after phone registration.
- Preserve existing email/Google authentication if it is already used by staff or existing accounts.

Do not log OTPs or store them in plaintext.

3.3 Food and restaurant discovery

Create a polished browse-first food experience.

The food screen must include:

- Search for food, groceries, drinks, and restaurants
- Ethiopian and Somali cuisine filters
- Category chips such as Pizza, Breakfast, Lunch, Dinner, Drinks, Bakery, Desserts, Grocery, Fresh Produce, and Dairy
- Top-rated restaurants
- Popular local vendors
- Featured items and offers
- Vendor open/closed status
- Vendor rating
- Estimated delivery time
- Distance
- Delivery fee
- Minimum order value
- Search aliases in English, Amharic, and Somali where practical
- Filter and Sort controls

Filters must support:

- Rating
- Delivery time
- Price

- Distance
- Availability/open status
- Promotion
- Cuisine/category

Do not invent discounts. All offers and discount values must come from real configured data.

3.4 Vendor storefront

Improve vendor pages so a customer can make a decision quickly.

Display:

- Vendor name, logo, and cover image
- Rating and review count
- Cuisine/business type
- Open/closed status
- Operating hours
- Delivery ETA
- Distance
- Delivery fee
- Minimum order
- Menu/product categories
- Product images, descriptions, prices, availability, and preparation times
- Product options and add-ons where supported
- Vendor policies and contact/support entry point

3.5 Real multi-item cart

Replace the current primarily single-product checkout behavior with a proper cart model.

Requirements:

- Add multiple products to a cart.
- Update quantity.
- Remove items.
- Support product options/add-ons.
- Show vendor grouping.

- Clearly decide and enforce one of these rules:
  - One vendor per cart for MVP, or
  - Multiple vendor cart that creates separate linked orders.
- Persist the cart for authenticated users for a configurable period.
- Revalidate product price, availability, vendor status, and quantity before order creation.
- Prevent duplicate orders on repeated taps or network retries.
- Show a clear empty-cart state.

Do not force the new cart into the existing single-batch `Order` shape without designing an appropriate `Cart`, `CartItem`, `OrderGroup`, and `OrderItem` model.

3.6 Checkout and pricing

Create a transparent, server-authoritative checkout summary.

Display separate line items for:

- Product subtotal
- Add-ons
- Discount
- Coupon
- Delivery fee
- Service fee
- VAT/tax
- Tip
- Wallet/points redemption, if enabled
- Final total

Requirements:

- Calculate final prices server-side or inside a trusted transaction.
- Never trust client-submitted totals.
- Show the final amount before confirmation.
- Support Cash on Delivery where enabled.
- Add a payment-provider abstraction for future Telebirr, CBE Birr, eBirr, and other providers.
- Handle payment success, failure, cancellation, pending, timeout, and refund states.

- Use idempotency keys for order and payment creation.
- Record every payment attempt and provider reference.
- Use signed/authenticated payment webhooks.
- Preserve current financial accounting and merchant payout rules.

3.7 Delivery serviceability

Improve location handling.

Requirements:

- Support map pin, search, GPS permission, Plus Code, and manual landmark/address entry.
- Store coordinates and normalized address.
- Create configurable delivery zones using polygons or approved radius rules.
- Display a clear “Service not available in this area” state.
- Prevent checkout outside a serviceable zone.
- Calculate delivery fee and ETA using zone, distance, vendor, order value, and configured rules.
- Allow admins to activate/deactivate zones without code changes.

3.8 Orders and history

Provide a simple customer-facing order experience while keeping internal operational statuses.

Customer-facing statuses should be understandable:

- Order placed
- Vendor confirmed
- Preparing
- Ready for pickup
- Courier assigned
- Picked up
- Out for delivery
- Delivered
- Canceled
- Refund pending

- Refunded

Keep internal statuses such as `RUNNER_ASSIGNED` and `RIDER_ACCEPTED` for operations, but map them to customer-friendly labels.

History must be grouped or filterable by:

- Food
- Delivery
- Ride
- Hotel
- Electricity
- Marketplace
- Other services

Allow reorder using current prices and current availability.

3.9 Ride and booking guardrails

Add protection against conflicting active bookings.

At minimum:

- Prevent a second active ride if the customer already has an ongoing ride.
- Prevent duplicate parcel bookings caused by repeated submission.
- Show a clear message explaining the conflict and link to the active booking.
- Apply the same idempotency and active-booking logic server-side.

3.10 Hotel booking

Add hotel booking as a separate bounded service, not as a food-order variant.

MVP hotel capabilities:

- Search by city
- Hotel cards with images, rating, location, and price range
- Hotel detail page
- Amenities such as Wi-Fi, air conditioning, Smart TV, prayer room, and security
- Room types and prices
- Availability count
- Guest details

- Booking dates
- Reservation confirmation
- Cancellation and refund rules
- Reviews and ratings

Use separate models such as `Hotel`, `RoomType`, `Availability`, `Reservation`, `Guest`, and `HotelReview`.

3.11 Loyalty points

Add a cross-service loyalty system.

Requirements:

- Show points balance and membership tier.
- Earn points after completed and non-refunded orders or bookings.
- Support configurable earning rules by service.
- Show progress toward the next tier.
- Support redemption only through server-side rules.
- Reverse or freeze points after cancellation or refund.
- Maintain an immutable loyalty ledger.

Do not use the wallet balance as a replacement for the points ledger.

3.12 Help Center and support

Create a simpler customer-facing Help Center while preserving existing chat and WhatsApp capabilities.

Flow:

1. Select service: Food, Parcel, Ride, Hotel, Electricity, or Other.
2. Select issue category.
3. Select related order or booking.
4. Enter description.
5. Attach photo, receipt, or evidence when appropriate.
6. Submit ticket.
7. Track ticket status.
8. Escalate to chat or WhatsApp when necessary.

Support agents must see only data allowed by their role. Sensitive payment, identity, and receipt data must remain protected.

3.13 Language and localization

Complete and consistently wire English, Somali, and Amharic throughout the customer experience.

All of the following must be localized:

- Navigation
- Buttons
- Validation errors
- OTP screens
- Checkout
- Order statuses
- Notifications
- Help Center
- Loyalty labels
- Hotel and service categories

Avoid hardcoded customer-facing English strings in new components. Use the existing locale infrastructure or improve it centrally.

---

4. Technical and Security Requirements

1. Preserve and improve Firestore rules.
2. Enforce authorization server-side.
3. Validate every role transition and order status transition.
4. Use immutable order snapshots for product names, prices, fees, taxes, addresses, and payment method.
5. Use server timestamps where possible.
6. Never trust client-side totals, role values, payment confirmation, or delivery completion.
7. Add audit events for money, access, role, order-status, refund, loyalty, and inventory actions.
8. Use idempotency for OTP requests, payment attempts, cart checkout, ride creation, parcel creation, and refunds.

9. Protect identity documents, receipts, phone numbers, and support attachments.
10. Add rate limits for OTP, auth, search abuse, coupons, support, and payment endpoints.
11. Ensure only assigned riders can submit delivery proof or restricted custody updates.
12. Preserve the EEU custody state machine and receipt security.
13. Do not add hardcoded admin allowlists for new roles or new production features.
14. Do not put secrets, API keys, or private URLs in client code.
15. Use feature flags for incomplete services such as Hotel and online payments.

---

5. UX and Visual Direction

Create an original visual identity for Jijiga Express:

- Emerald/teal for trust, delivery, and logistics
- Amber/orange for food and offers
- Blue for parcel and maps
- Violet for admin and analytics
- White and slate neutrals for clarity

Use:

- Large readable type
- Strong contrast
- Rounded but professional cards
- Thumb-friendly controls
- Clear empty, loading, offline, and error states
- Accessible tap targets
- Responsive layouts for Android phones first
- Smooth but restrained transitions
- No excessive gradients, emoji dependence, or decorative elements that reduce clarity

The product should feel local, reliable, fast, and operationally trustworthy.

---

6. Implementation Process

Follow this process in order:

Phase 1: Inspect and plan

- Inspect the repository structure, routes, components, services, Firestore rules, and tests.
- Identify existing code that can be reused.
- Create or update `plan.md` with architecture, data-model changes, UI changes, security implications, and migration strategy.
- Create or update `TODO.md` with concrete deliverables and acceptance criteria.
- Do not start by deleting or replacing major files.

Phase 2: Foundation

Implement:

- Centralized design tokens
- Mobile bottom navigation
- Home service grid
- Unified settings and Help Center entry points
- Locale wiring for English, Somali, and Amharic
- Customer-friendly order-status mapping
- Feature flags for incomplete services

Phase 3: Ordering core

Implement:

- Phone OTP flow
- Serviceability zones
- Real cart model
- Multi-item or one-vendor cart rules
- Server-side checkout calculation
- Idempotent order creation
- Customer order history grouping

Phase 4: Growth services

Implement:

- Loyalty ledger and tiers
- Hotel booking

- Improved food filters and sorts
- Vendor ratings/reviews
- Favorites and reorder
- Payment-provider abstraction

Phase 5: Validation

Run and fix:

- TypeScript typecheck
- Production build
- Unit tests
- Firestore security-rule tests
- Cart and checkout integration tests
- Payment idempotency tests
- Role/authorization tests
- Mobile responsive checks
- Accessibility checks

Do not claim a feature is complete unless it works in the UI, data model, authorization layer, and tests.

---

7. Required Deliverables

At the end of the implementation:

1. Updated source code.
2. Updated Firestore rules and indexes if required.
3. Updated types and migrations.
4. Updated localization files.
5. Updated tests.
6. Updated `plan.md` and `TODO.md`.
7. A short `IMPLEMENTATION_REPORT.md` containing:
    - Features implemented
    - Features intentionally deferred

- Security changes
- Data migrations required
- Test results
- Known limitations
- Deployment steps

8. A production build that passes.
9. A clean, downloadable project archive excluding `node_modules`, build caches, secrets, and generated local artifacts.

---

8. Acceptance Criteria

The upgrade is acceptable only if all of the following are true:

1. A new customer can register using a valid Ethiopian or Somaliland phone number and OTP.
2. Customer self-registration cannot create privileged roles.
3. A customer can select a location and receive a serviceability result.
4. A customer can browse categories, search products/vendors, and sort/filter results.
5. A customer can add multiple items to a valid cart and see an accurate checkout breakdown.
6. The server rejects tampered totals, invalid coupons, unavailable inventory, and unsupported addresses.
7. Repeated checkout taps do not create duplicate orders.
8. A vendor can receive and process a new order.
9. A delivery worker can accept, pick up, and complete a delivery according to valid status transitions.
10. A customer can see friendly order statuses and live tracking where available.
11. A customer can view service-grouped history and reorder with current availability.
12. A customer cannot create a second active ride when one already exists.
13. A customer can submit a categorized support ticket linked to an order or booking.
14. English, Somali, and Amharic customer-facing strings work in the core flows.
15. Existing EEU, parcel, ride, inventory, accounting, and security regressions remain passing.

16. Typecheck and production build pass without unresolved errors.
17. Tests cover authentication, authorization, cart calculation, price tampering, idempotency, inventory, payment states, loyalty reversal, and order transitions.

Start by inspecting the existing repository and reporting the current architecture and gaps. Then implement the highest-value changes incrementally, beginning with the customer foundation and ordering core. Do not rewrite the project blindly.